C++ Numeric Types, Overflow and Floating Point

A practical reference with measured examples

- Part 1 — Integer Types
  - 1.1 Sizes and ranges
  - 1.2 Unsigned overflow — defined, and still a bug source
  - 1.3 Signed overflow — undefined behaviour
  - 1.4 Integer promotion — the silent one
  - 1.5 Overflow before widening
  - 1.6 Shift traps
  - 1.7 Conversion and truncation
  - 1.8 Practical rules
- Part 2 — Floating Point
  - 2.1 How a double is laid out
  - 2.2 Error source 1 — representation
  - 2.3 Error source 2 — absorption
  - 2.4 Error source 3 — non-associativity
  - 2.5 Error source 4 — catastrophic cancellation
  - 2.6 Error source 5 — accumulation
  - 2.7 Comparing floats correctly
  - 2.8 Special values
  - 2.9 FMA and compiler flags
  - 2.10 Rounding modes
  - 2.11 Fixed point — the alternative
  - 2.12 Decision checklist

Part 1 — Integer Types

1.1 Sizes and ranges

Measured on x86-64 Linux, GCC 13. **Only the minimums are guaranteed by the standard** — the actual values below are the LP64 model used by Linux, macOS and most Unix systems.

Type	Bytes	Bits	Signed	Min	Max
`bool`	1	8	no	0	1
`char`	1	8	**implementation-defined**	−128	127
`signed char`	1	8	yes	−128	127
`unsigned char`	1	8	no	0	255
`short`	2	16	yes	−32,768	32,767
`unsigned short`	2	16	no	0	65,535
`int`	4	32	yes	−2,147,483,648	2,147,483,647
`unsigned int`	4	32	no	0	4,294,967,295
`long`	8	64	yes	-9.22×10^{18}	9.22×10^{18}
`long long`	8	64	yes	-9.22×10^{18}	9.22×10^{18}
`unsigned long long`	8	64	no	0	1.84×10^{19}
`size_t`	8	64	**no**	0	1.84×10^{19}
`ptrdiff_t`	8	64	yes	-9.22×10^{18}	9.22×10^{18}
`__int128`	16	128	yes	$\approx -1.7\times10^{38}$	$\approx 1.7\times10^{38}$

The portability traps

`long` is not portable. 64 bits on Linux/macOS (LP64), **32 bits on Windows** (LLP64). `long long` is at least 64 everywhere. If you need exactly 64 bits, say `int64_t`.

`char` signedness is implementation-defined. Signed on x86 Linux, **unsigned on ARM Linux**. If you care, write `signed char` or `unsigned char` explicitly.

Only minimums are guaranteed: `char` ≥ 8 bits, `short` ≥ 16, `int` ≥ 16 (yes, 16), `long` ≥ 32, `long long` ≥ 64.

Use `<cstdint>` when the width matters:

Family	Meaning
`int8_t` … `int64_t`	exactly N bits (may not exist on exotic platforms)
`int_least8_t` …	at least N bits, smallest such type
`int_fast8_t` …	at least N bits, fastest such type
`intmax_t`	largest supported integer
`intptr_t`	can hold a pointer

1.2 Unsigned overflow — defined, and still a bug source

Unsigned arithmetic is **modulo 2^n**. Well-defined, never UB.

```
0u - 1              = 4294967295
UINT_MAX + 1        = 0
(size_t)0 - 1       = 18446744073709551615
```

The loop-backwards bug

This is the single most common unsigned bug in C++:

```
        for (size_t i = v.size() - 1; i >= 0; --i) { ... }   // BROKEN
```

Two failures at once:

1. If `v` is **empty**, `v.size() - 1` is `18446744073709551615`, so the first access is wildly out of bounds.
2. `i >= 0` is **always true** for an unsigned type, so it never terminates.

Correct forms:

```
for (size_t i = v.size(); i-- > 0; ) { ... }          // post-decrement idiom
for (auto it = v.rbegin(); it != v.rend(); ++it) { }  // reverse iterator
```

Where this bit in the assessment

The ring buffer index:

```
buf[(head + size - count + i) % size];
```

The `+ size` is **load-bearing**. `head - count` can go "negative" on `size_t`, wrapping to ~1.8×10^{19}, and the modulo then produces a **valid-looking but wrong index**. Silent corruption, not a crash — the worst kind.

1.3 Signed overflow — undefined behaviour

```
INT_MAX + 1   → UB (typically wraps to INT_MIN, but you cannot rely on it)
-INT_MIN      → UB (no positive counterpart in two's complement)
abs(INT_MIN)  → UB for the same reason
INT_MIN / -1  → UB
```

Why "it wraps in practice" is not good enough: the compiler optimises *assuming overflow never happens*. So:

```
if (x + 1 < x) { /* overflow check */ }   // DELETED by the optimiser
```

The compiler reasons: signed overflow is UB, so `x + 1 < x` can never be true, so this branch is dead. It removes your check entirely. Same with `x * 2 / 2` → simplified to `x`.

How to check safely:

```
// GCC/Clang builtins — preferred
int result;
if (__builtin_add_overflow(a, b, &result)) { /* handle */ }

// Or pre-check before doing the operation
if (a > INT_MAX - b) { /* would overflow */ }

// C++20
#include <numeric>   // std::midpoint avoids (a+b)/2 overflow
```

Compiler flags: `-fwrapv` makes signed overflow wrap (defined), `-ftrapv` traps at runtime, `-fsanitize=signed-integer-overflow` (part of UBSan) catches it in testing. Use UBSan in CI.

1.4 Integer promotion — the silent one

Anything smaller than `int` is promoted to `int` before arithmetic.

```
unsigned char 200 + unsigned char 200  = 400    (int arithmetic, no wrap)
stored back into unsigned char          = 144    (now it wraps)

short 30000 + short 30000               = 60000  (int arithmetic)
stored back into short                  = -5536  (now it overflows)
```

So the overflow happens at the **assignment**, not at the operation. That's why the intermediate result looks correct in a debugger.

The usual arithmetic conversions

When operand types differ, they're brought to a common type by these rules, in order:

1. Anything smaller than `int` → `int`
2. If both are signed or both unsigned → the larger rank wins
3. **If the unsigned type has rank ≥ the signed type → the signed operand converts to unsigned**
4. Otherwise, if the signed type can represent all values of the unsigned → unsigned converts to signed
5. Otherwise both convert to the unsigned version of the signed type

Rule 3 is the one that bites:

```
int si = -1;
unsigned ui = 1;
si < ui                 // FALSE — -1 becomes 4294967295
```

Measured: `int(-1) < unsigned(1)` evaluates to **false**.

Always compile with `-Wsign-compare` (included in `-Wall`). It catches exactly this.

1.5 Overflow before widening

```
int big = 100000;
long long wrong = big * big;              // 1410065408   <- WRONG
long long right = (long long)big * big;   // 10000000000  <- correct
```

The multiplication happens in `int` and overflows **before** the assignment widens it. The target type does not influence the arithmetic.

Fix: cast **one operand** before the operation. Casting both is redundant but harmless.

This pattern appears constantly in financial code: `qty * price` where both are `int`.

1.6 Shift traps

Expression	Status
`1 << 31`	**UB** for signed `int` — shifts into the sign bit
`1u << 31`	fine — 2147483648
`1 << 32`	**UB** — shift count ≥ width
`x << -1`	**UB** — negative shift
`-1 >> 1`	implementation-defined (arithmetic shift in practice)
`1LL << 40`	fine — needs the `LL` suffix

The `1 << 40` mistake is common: the literal `1` is `int`, so the shift is UB regardless of what you assign it to.

1.7 Conversion and truncation

Narrowing silently truncates:

```
int64_t big = 5000000000;
int32_t small = big;          // truncated, implementation-defined pre-C++20
```

Since C++20, converting an out-of-range value to a signed type is defined as modular (two's complement), but it's still almost certainly a bug.

Float to int truncates toward zero and is **UB if out of range**:

```
(int)3.9    → 3
(int)-3.9   → -3
(int)1e20   → UB
```

Use `std::lround`, `std::floor`, `std::ceil` when you mean rounding, and range-check first.

Brace initialisation catches narrowing at compile time:

```
int x{3.5};       // ERROR: narrowing conversion
int y = 3.5;      // silently truncates to 3
```

That alone is a decent reason to prefer `{}` initialisation.

1.8 Practical rules

1. **Use `int64_t` for anything that accumulates.** Positions, quantities, tick counts. `int` gives you 2.1 billion, which sounds like a lot until you count messages.
2. **Prefer signed for arithmetic**, even for values that can't be negative. Unsigned wraparound in subtraction is a worse failure mode than a negative number you can assert on. (This is the Google style guide position; the STL disagrees by using `size_t`, which is widely regarded as a historical mistake.)
3. **Never mix signed and unsigned** in a comparison. Turn on `-Wsign-compare`.
4. **Cast before multiplying** when the result could exceed the operand type.
5. **Run UBSan in CI** — `-fsanitize=undefined` catches signed overflow, bad shifts and out-of-range float-to-int conversions, at very low cost in a test build.

Part 2 — Floating Point

2.1 How a double is laid out

IEEE-754 binary64:

```
| sign | exponent  | mantissa (fraction)             |
|  1   |    11     |               52                |
```

The value is `(−1)^sign × 1.mantissa × 2^(exponent − 1023)`.

The leading `1.` is **implicit** — not stored — which buys one extra bit of precision. That's why `digits` reports 53 while only 52 bits are stored.

Type	Bytes	Mantissa bits	Decimal digits	Min normal	Max	Epsilon
`float`	4	24	6	1.18×10^{-38}	3.40×10^{38}	1.19×10^{-7}
`double`	8	53	15	2.23×10^{-308}	1.80×10^{308}	2.22×10^{-16}
`long double`	16	64	18	—	—	1.08×10^{-19}

`long double` on x86 is the 80-bit x87 format padded to 16 bytes. It's **not** portable — on MSVC and ARM it's usually just a `double`.

Epsilon is the gap between 1.0 and the next representable double. It is *not* a universal error bound — see §2.7.

2.2 Error source 1 — representation

Binary can only represent fractions whose denominator is a power of two. `0.1` is a repeating fraction in binary, exactly as `1/3` is in decimal.

```
0.1  stored as  0.1000000000000000555
0.2  stored as  0.2000000000000001110
0.3  stored as  0.2999999999999998890

0.1 + 0.2    =  0.3000000000000004441
0.1 + 0.2 == 0.3  →  FALSE
```

In hex, where the difference is one bit:

```
0.1 + 0.2  =  0x1.3333333333334p-2
0.3        =  0x1.3333333333333p-2
```

Consequence: never compare computed floats with `==`. But note the converse — comparing a value you *copied* is fine, which is why bit-parity tests use exact `==` or `.hex()` deliberately.

2.3 Error source 2 — absorption

Adding a small number to a large one can lose the small number entirely, because the result has to fit in 53 bits of mantissa.

```
1e16 + 1        = 10000000000000000     (the 1 vanished)
1e16 + 1 == 1e16 → TRUE
```

The integer limit: doubles represent every integer exactly up to $\mathbf{2^{53} = 9{,}007{,}199{,}254{,}740{,}992}$. Above that, they start skipping:

```
2^53      = 9007199254740992
2^53 + 1  → stored as 9007199254740992   (rounded down — the odd number is unrepresentable)
```

This is why JavaScript, which has only doubles, has a `BigInt` type — and why you must never store a 64-bit ID in a double.

The gap grows with magnitude

Measured spacing between adjacent doubles:

Near	Gap to next double
1	2.2×10^{-16}
10^{6}	1.2×10^{-10}
10^{9}	1.2×10^{-7}
10^{12}	1.2×10^{-4}
10^{15}	0.125
10^{16}	**2.0**
10^{18}	128

Near 10^{16} the *smallest possible increment is 2*. Precision is **relative**, not absolute — you always get ~15–16 significant digits, never a fixed number of decimal places.

Practical implication: a rupee amount stored as a double is fine at ₹1,000.00 and meaningless at ₹10,000,000,000,000,000.

2.4 Error source 3 — non-associativity

Floating-point addition is **not associative**. Same numbers, different grouping, different answer:

```
a = 1e16, b = -1e16, c = 1.0

(a + b) + c  =  1.0     (a+b is exactly 0, then +1)
a + (b + c)  =  0.0     (b+c absorbs the 1, then a+b is 0)
```

Consequences:

- The compiler may **not** reassociate FP arithmetic — unless you pass `-ffast-math`, which permits it and makes your results flag-dependent.
- **Parallel reductions are non-deterministic.** Summing an array across N threads gives a different answer depending on how the work was split and which thread finished first. If you need reproducibility, fix the reduction tree or sum serially.

2.5 Error source 4 — catastrophic cancellation

Subtracting two nearly equal numbers destroys the significant digits, leaving only the noise.

```
x = 1.0
y = 1.0 + 1e-12

y - x  = 1.000089e-12    (true value 1e-12)
relative error = 8.9e-05
```

You lost **four digits of accuracy in a single subtraction**, because the leading digits cancelled and what remains is the rounding error from constructing `y`.

The classic example: the quadratic formula

For $x^2 + 10^8 x + 1 = 0$:

```
naive:   (-B + sqrt(B*B - 4AC)) / 2A   =  -7.450580596923828e-09
stable:  2C / (-B - sqrt(B*B - 4AC))   =  -1.000000000000000e-08
```

The naive form is **wrong in the second significant digit**. When B is large, $\sqrt{B^2-4AC} \approx B$, so $-B + \text{that}$ is catastrophic cancellation. The stable form multiplies numerator and denominator by the conjugate to avoid the subtraction.

The general lesson: rearrange the algebra to avoid subtracting near-equal quantities. There is often an equivalent formula that doesn't.

2.6 Error source 5 — accumulation

Errors compound over long sums. Measured, adding `0.1` ten million times:

```
naive sum          = 999999.999839    (true 1000000)
absolute error     = 1.61e-4
Kahan summation    = 1000000.000000   error 0.00e+00
```

Kahan summation

Tracks the lost low-order bits in a compensation variable and feeds them back:

```
double sum = 0.0, c = 0.0;
for (double x : values) {
    double y = x - c;        // recover what was lost last time
    double t = sum + y;      // this addition loses low bits
    c = (t - sum) - y;       // capture exactly what was lost
    sum = t;
}
```

Roughly 4× the arithmetic, but error becomes O(1) instead of O(n).

Alternatives: pairwise summation (what NumPy does — error O(log n), nearly free), or sorting by magnitude before summing.

Where this appeared in the assessment

An average price maintained by repeated blending drifted from `0.3333333333333333` to `0.33333333333395226` over 200,000 fills. The fix wasn't Kahan — it was **changing the state variable**: store an exact integer *cost basis* and derive the average on demand. Integers don't drift.

That's the more general lesson: if you're fighting accumulation error, ask whether you're storing the wrong thing.

2.7 Comparing floats correctly

Absolute tolerance — only valid when you know the scale:

```
std::fabs(a - b) < 1e-9       // meaningless if a, b are ~1e15
```

Relative tolerance — scales with magnitude:

```
std::fabs(a - b) <= tol * std::max(std::fabs(a), std::fabs(b));
```

Combined, which is what you usually want (handles values near zero, where relative fails):

```
bool nearly_equal(double a, double b, double rel = 1e-9, double abs_tol = 1e-12) {
    double diff = std::fabs(a - b);
    if (diff <= abs_tol) return true;                    // handles a,b ≈ 0
    return diff <= rel * std::max(std::fabs(a), std::fabs(b));
}
```

ULP comparison — count representable values between them. The most rigorous, and what you want for numerical library testing:

```
#include <cmath>
// how many doubles lie between a and b
```

`std::nextafter` walks one step; comparing bit patterns as integers gives ULP distance directly.

Exact comparison is correct when the values were *copied* rather than computed — a serialisation round-trip, or validating a reimplementation. That's why parity tests compare `.hex()`.

2.8 Special values

```
1.0 / 0.0    =  inf
-1.0 / 0.0   = -inf
0.0 / 0.0    =  nan
inf - inf    =  nan
```

NaN — the rules that matter

Every ordered comparison involving NaN is false, including `NaN == NaN`.

```
nan == nan   → false
nan != nan   → TRUE      ← the standard NaN test
nan <  1.0   → false
nan >  1.0   → false
nan <= 1.0   → false
```

Two direct consequences:

```
        if (!(price > 0.0))   // catches zero, negatives, AND NaN
        if (price <= 0.0)     // MISSES NaN, because that's also false
```

```
        if (!(std::fabs(r) <= band))   // excludes NaN
        if (std::fabs(r) > band)       // ADMITS NaN
```

Each pair is equivalent for normal numbers and **opposite** for NaN. Choosing deliberately is the whole skill.

NaN in a sort comparator is UB, not just a bad order — it breaks strict weak ordering, because `a` and `NaN` look "equivalent", `b` and `NaN` look "equivalent", but `a` and `b` are not. Transitivity of equivalence fails. Measured effect: 72 NaNs among 500 elements left **25 perfectly valid entries out of order**.

NaN propagates silently. Any arithmetic touching it produces NaN, and every comparison is false — so `if (price > limit)` quietly stops firing. That's worse than a crash.

Infinity is more dangerous than NaN in ranking code

Infinity **compares consistently**. So a `+inf` return sorts cleanly to rank 1 — no crash, no warning, and the signal ships. In the assessment, a stock with a zero prior close became "today's top gainer".

Signed zero

```
0.0 == -0.0   → true
1 / 0.0       →  inf
1 / -0.0      → -inf
```

Equal under comparison, different in behaviour. Also `std::signbit(-0.0)` is true.

Denormals (subnormals)

Numbers below the smallest normal (2.2×10^{-308} for double), down to 4.9×10^{-324}. They fill the gap around zero at reduced precision.

Performance cliff: on many CPUs, operations on denormals trigger a microcode assist costing **~100 cycles**. In a hot loop that's catastrophic and invisible in the source.

Mitigate with flush-to-zero (FTZ/DAZ) via `_MM_SET_FLUSH_ZERO_MODE`, or by not producing them — fixed point removes the case entirely.

Classification

```
        std::isnan(x)      std::isinf(x)      std::isfinite(x)
        std::isnormal(x)   std::signbit(x)    std::fpclassify(x)
```

2.9 FMA and compiler flags

A **fused multiply-add** computes `a*b + c` with a **single rounding** instead of two. It is faster *and* more accurate — which is exactly why it breaks bit-for-bit reproduction of a two-step computation.

Measured, 3,000 fills against a Python reference:

Build	Fills differing
`-O0`	0
`-O2`	0
`-O2 -ffp-contract=off`	0
`-O0 -march=native`	0
`-O2 -march=native`	**1,781 (59%)**
`-Ofast -march=native`	1,659

The generated assembly:

```
-O2                                mulsd, mulsd, addsd      (two roundings)
-O2 -march=native                  vmulsd, vfmadd132sd      (ONE rounding)
-O2 -march=native -ffp-contract=off  vmulsd, vmulsd, vaddsd   (two again)
```

The subtlety: GCC's default is already `-ffp-contract=fast`. Plain `-O2` passes only because **baseline x86-64 has no FMA instruction** — the compiler is permitted to fuse but has nothing to fuse with. `-march=native` makes it available. And **FMA is baseline on ARMv8**, so the same source contracts by default on Graviton or Apple Silicon.

So bit-parity survived on two coincidences — target ISA and optimisation level — neither of which anyone chose deliberately.

Flag reference

Flag	Effect
`-ffp-contract=off`	no FMA fusion — required for bit-exact reproduction
`-ffp-contract=fast`	**GCC default** — fusion permitted
`-ffast-math`	bundle: no NaN/inf assumed, reassociation allowed, denormals flushed, reciprocals approximated. **Breaks IEEE.**
`-Ofast`	`-O3` plus `-ffast-math`
`-frounding-math`	tells the compiler the rounding mode may change at runtime
`-fexcess-precision=standard`	avoids x87 80-bit intermediate surprises on 32-bit x86

Never use `-ffast-math` where reproducibility or NaN handling matters. It also silently disables your `isnan` checks, because the compiler now assumes NaN can't occur.

2.10 Rounding modes

IEEE-754 defines four. The default is **round-to-nearest, ties-to-even** — so `0.5` rounds to `0`, `1.5` rounds to `2`. Ties-to-even avoids the upward bias of always rounding half up.

```
#include <cfenv>
std::fesetround(FE_UPWARD);      // FE_DOWNWARD, FE_TOWARDZERO, FE_TONEAREST
```

Changing it is rare outside interval arithmetic, and note that most compilers assume the default unless you pass `-frounding-math`.

2.11 Fixed point — the alternative

When values are exact multiples of a known unit — prices in ticks, currency in paise — integers are strictly better:

```
constexpr int64_t SCALE = 10000;                    // 4 decimal places
int64_t ticks = std::llround(rupees * SCALE);
```

	Floating point	Fixed point
Addition / subtraction	rounds, accumulates drift	**exact**
Multiplication	rounds	exact if you manage scale
Division	rounds	rounds — you choose where the remainder goes
NaN / inf / denormals	all present	**none exist**
Speed	fast	**faster**
Range at 1e4 scale on int64	$\pm 1.8\times10^{308}$	$\pm 9.2\times10^{14}$

Costs: you manage the scale yourself, so a units error becomes a silent 10,000× rather than a visible one. Multiplication needs a wider intermediate (`__int128`) to avoid overflow before you rescale.

Where it pays: the assessment's corrected position tracker stores an exact integer cost basis and *derives* the average price. That gives an exact conservation property — realised plus remaining cost equals cash paid, at every point, provable — which a floating implementation cannot state at all.

2.12 Decision checklist

Use `double` when: the quantity is genuinely continuous (a rate, a vol, a statistical result), or you need the enormous dynamic range.

Use `int64_t` fixed point when: values are exact multiples of a unit (money, tick-quantised prices), values accumulate over a long session, or you need exact reproducibility and provable conservation.

Use `float` when: memory bandwidth dominates and 7 digits suffice (large ML tensors, graphics). Rarely correct for finance.

Use `__int128` when: you need an exact intermediate for fixed-point multiplication.

Quick rules

1. Never compare computed floats with `==`.
2. `!(x > 0)` and `x <= 0` differ on NaN. Choose deliberately.
3. Doubles hold exact integers only up to 2^{53}.
4. Never store money as a `float`.
5. Set `-ffp-contract=off` if you need bit-exact reproduction; never use `-ffast-math` there.
6. Parallel FP reductions are non-deterministic unless you fix the tree.
7. Watch for cancellation — rearrange the algebra rather than adding tolerance.
8. If you're fighting accumulation drift, you're probably storing the wrong state variable.